

Informe de evaluación ciega independiente

Objeto evaluado: dos paquetes anónimos (candidate-x, candidate-y) que implementan el mismo MVP: *Business Intelligence Copilot para boxes de CrossFit*, según EVALUATION_RUBRIC.md y EVALUATOR_PROMPT.md.

Método: verificación de hashes, instalación y ejecución real de cada candidato siguiendo únicamente sus propias instrucciones, ejecución de sus baterías de test, y una batería de pruebas adversariales equivalente diseñada y ejecutada por el evaluador (fuera de ambos paquetes, en eval_workspace/adversarial/). No se ha usado contexto de conversaciones previas ni se ha intentado inferir qué modelo, herramienta o runtime generó cada candidato.

Este informe es autocontenido: no requiere acceso a ninguna conversación anterior para entenderse.

0. Verificación de integridad

```
sha256sum -c candidate-x-files.sha256 → 35/35 ficheros OK
sha256sum -c candidate-y-files.sha256 → 40/40 ficheros OK
```

Ambos paquetes coinciden byte a byte con sus manifiestos. Los ficheros de experiment/input/ (brief, historias de usuario, criterios de aceptación, PDF de la API, prompt y protocolo del experimento) tienen **hash idéntico** en ambos candidatos, confirmando que ninguno modificó la entrada congelada del experimento (no se activa el hard failure #4 en ninguno de los dos).

1. Candidate-X

Resultado de hard failures: PASS (ninguno de los 8 se activa)

- No hay llamadas de red, tokens, credenciales ni variables de entorno en el código (grep exhaustivo sobre src/ y scripts/).
- Ninguna cifra de negocio depende de un LLM: src/metrics/analytics.js son funciones puras verificadas por inspección y por ejecución.
- Corre con Node.js puro, sin dependencias (package.json no declara ninguna), sin credenciales.
- Las 5 preguntas del MVP se responden y todas tienen verificación automatizada (test/metrics.test.js, test/acceptance.test.js).
- Instalación y ejecución reproducidas con éxito exactamente como indica el README (npm start, npm test, npm run generate:data).

Puntuación por dimensión (sobre 100)

Dimensión	Puntos	Máximo
1. Corrección funcional	24	25
2. Determinismo y reproducibilidad	15	15
3. Arquitectura y sustituibilidad	14	15
4. Explicabilidad y prudencia	9	10
5. Comprensión de la API y trazabilidad	9	10
6. Calidad de la verificación automatizada	8	10
7. Documentación y handoff	9	10

Dimensión	Puntos	Máximo
8. Proporcionalidad y mantenibilidad	4	5
Total	92	100

Evidencia clave (reproducida por el evaluador, no solo declarada)

- `npm test` → **28/28 tests en verde** (confirmado ejecutando yo mismo, no solo leyendo el informe del candidato).
- Regeneración del dataset (`npm run generate:data`) produce un fichero **byte a byte idéntico** al committeado (`diff` vacío). Semilla y fecha ancla fijas; `metadata.asOf = 2026-06-30` se usa como referencia temporal, nunca el reloj del sistema — confirmado leyendo `copilot.js` (el `asOf` viene de `dataset.metadata.asOf`, no de `Date.now()`).
- Batería adversarial propia del evaluador (`x_adversarial.mjs`, 19 casos): **19/19 en verde**, incluyendo dataset vacío, periodo sin sesiones, capacidad cero (excluida y declarada explícitamente en `exclusions`), reservas canceladas y en lista de espera (excluidas correctamente de la ocupación), asistencia ligada a una sesión cancelada por el box (excluida tanto de ocupación como de “última asistencia”), cero explícito (0% distinguido de “sin datos”), periodo ambiguo (pide aclaración, no asume), dos periodos sin verbo de comparación (no se trata como comparación silenciosa), comparación de una métrica no soportada — cancelaciones en vez de ocupación — (el parser detecta la mezcla de intents y pide aclaración en vez de responder mal calladamente), parafraseos de las historias de usuario, orden determinista en 5 ejecuciones repetidas.
- Un hallazgo propio adicional: un usuario que reserva cada semana hasta 2 días antes de la fecha de análisis pero **nunca asiste realmente** (todo `attended:false`, sin cancelar) sí es marcado como inactivo, porque la definición de X se basa en la última **asistencia real**, no en la última reserva. Esto casa exactamente con el literal de la pregunta MVP #4 (“usuarios sin *asistir*”).
- Prueba manual con rango de fechas explícito fuera de los ejemplos del README (“...entre 2026-05-01 y 2026-05-31”) → responde correctamente sobre mayo, con métrica y exclusiones bien calculadas para ese periodo exacto.

Defectos confirmados

- Ninguno funcional. El único matiz: el mensaje de “pregunta no soportada” genérico (“La pregunta no corresponde a una de las cinco métricas del MVP.”) es bastante parco comparado con el caso especial de fidelidad-por-entrenador (que sí tiene una explicación rica de datos faltantes); para una pregunta fuera de dominio distinta de esa, la explicación es más pobre que la de Y. Es un matiz de calidad, no un defecto de corrección.

Afirmaciones no verificables o no comprobadas por el evaluador

- El propio candidato declara en `docs/06_ACCEPTANCE_EVALUATION.md` una progresión de “23 pruebas → 28 pruebas” durante el desarrollo; solo he podido verificar el estado final (28/28), no el historial intermedio. Es una afirmación de proceso, no de producto, y no afecta el veredicto.

Fortalezas

- Motor de métricas explícitamente puro, sin acoplarse al adaptador JSON; el propio `data-source.js` documenta el contrato como puerto reemplazable, y los tests ya corren la misma batería contra un adaptador JSON y contra una fuente en memoria.

- Validación de dataset muy estricta (`json-data-source.js`): rechaza referencias rotas, estados de reserva no soportados, fechas mal formadas, y una reserva marcada como asistida y cancelada a la vez.
- Documento de descubrimiento de API (`03_API_Discovery.md`) rico: matriz de capacidades, 7 contradicciones documentales identificadas con página exacta, y una sección explícita de “definiciones de negocio aprobadas para el mock” que separa con nitidez lo documentado de lo asumido.
- Dataset de fixture minúsculo y manejable (16 KB, 8 usuarios / 14 sesiones / 47 reservas) — suficiente para ejercitar cada regla de negocio sin ruido.

Riesgos residuales

- El parser de lenguaje natural es deliberadamente estrecho (por diseño, no por descuido): cualquier pregunta de negocio nueva necesitará ampliar `INTENT_PATTERNS` a mano.
- La brecha de integración real con AimHarder (enumeración completa de reservas, sin endpoint global) queda igual de abierta que en Y — es una limitación de la API real, no del candidato.
- La carpeta `docs/run/` (contexto, plan, checkpoint, informe de cierre) añade ~470 líneas de documentación de proceso que aportan poco al desarrollador que solo quiere mantener el producto; es documentación de *cómo se hizo*, no de *qué es y cómo se usa*.

Veredicto de reproducibilidad

Totalmente reproducible. Cero dependencias externas, instalación y ejecución triviales, dataset regenerable byte a byte, todos los tests y todos los ejemplos del README verificados de primera mano.

2. Candidate-Y

Resultado de hard failures: PASS (ninguno de los 8 se activa formalmente; ver matiz en el punto 8)

- Sin llamadas de red, tokens ni credenciales (`grep` exhaustivo sobre `src/` y `scripts/`).
- Métricas puras sin LLM, verificado por inspección y ejecución (`metrics.py`).
- Solo biblioteca estándar de Python + `pytest` como dependencia de test.
- Las 5 preguntas del MVP están implementadas y automáticamente testeadas.
- Instalación y ejecución reproducidas con éxito (entorno virtual, `pip install pytest`, `PYTHONPATH=src` `pytest`, CLI). Nota de entorno: `pyproject.toml` declara `requires-python >= 3.11`; el único intérprete disponible en el entorno de evaluación era 3.10.12, y el proyecto funcionó igualmente sin ningún error de sintaxis ni de biblioteca. Esto es una discrepancia de documentación (posiblemente conservadora) frente al entorno de prueba, no un fallo de ejecución del candidato: se cuenta como limitación de entorno, no como defecto de producto.

Puntuación por dimensión (sobre 100)

Dimensión	Puntos	Máximo
1. Corrección funcional	17	25
2. Determinismo y reproducibilidad	15	15
3. Arquitectura y sustituibilidad	13	15
4. Explicabilidad y prudencia	7	10

Dimensión	Puntos	Máximo
5. Comprensión de la API y trazabilidad	9	10
6. Calidad de la verificación automatizada	7	10
7. Documentación y handoff	8	10
8. Proporcionalidad y mantenibilidad	3	5
Total	79	100

Evidencia clave (reproducida por el evaluador)

- pytest → **57/57 tests en verde** (confirmado ejecutando yo mismo).
- Regeneración del dataset (generate_dataset.py) produce ficheros **byte a byte idénticos** a los committeados. Semilla fija (SEED=1337) y ANCHOR_DAY = date(2026,7,20) como constante fija en código — no depende del reloj del sistema, aunque coincide (por casualidad, verificado leyendo el código fuente) con la fecha real de hoy en este entorno de evaluación.
- Batería adversarial propia del evaluador (y_adversarial.py, 15 casos): **15/15 en verde** contra las aserciones planteadas, pero dos de esas comprobaciones revelan defectos de producto reales y no cubiertos por la suite de 57 tests del propio candidato:

Defecto confirmado 1 — capacidad cero se mezcla silenciosamente en la media de ocupación. Con una sesión de capacidad 0 y otra al 100 %, average_occupancy devuelve **50 %** sin ninguna exclusión ni limitación declarada (exclusions y limitations quedan vacíos para esa sesión). Candidate-x, ante el mismo caso, **excluye** la sesión de capacidad no positiva y lo declara explícitamente en exclusions. Esto es exactamente el caso “capacidad igual a cero” que pide la rúbrica, y viola el requisito de explicabilidad (AC-03: datos excluidos) de forma silenciosa — la cifra que ve el propietario del box está sesgada sin que se le avise.

Defecto confirmado 2 — un rango de fechas explícito e inequívoco en la pregunta se ignora en silencio. Pregunta: “¿Cuál fue la ocupación media entre 2026-05-01 y 2026-05-31?”. Candidate-y **ignora las fechas literales** de la pregunta, responde sobre “este mes” (el mes en curso, julio, en este entorno) y antepone el mensaje “No se especificó un periodo; asumo ‘este mes’.” — una afirmación falsa, porque sí se especificó un periodo, solo que el enrutador de lenguaje natural no reconoce el patrón YYYY-MM-DD. Verificado ejecutando la CLI real, no solo el motor de métricas. Candidate-x, con la pregunta equivalente, responde correctamente sobre mayo.

Ninguno de estos dos casos está cubierto por los 57 tests del candidato (grep sobre tests/*.py no encuentra ninguna prueba de capacidad 0 ni de rango de fechas explícito en nlp.py), pese a que la documentación de Y presume de una revisión de código adversarial explícita que corrigió 8 defectos de corrección — ninguno de los cuales es este par. Esto es evidencia directa y reproducible de que “tests en verde” no agotan la verificación de corrección, tal y como advierte la rúbrica.

- Divergencia semántica confirmada y documentada honestamente por el propio candidato: la métrica de “usuarios inactivos” se basa en **ausencia de reserva**, no en **ausencia de asistencia real**, porque el único endpoint de la API documentada que soporta esto directamente (GET /clients/no-booking/:date) mide reservas, no asistencia. Reproduce el caso concreto: un usuario que reserva cada semana hasta 2 días antes de la fecha de análisis pero nunca asiste (attended:false en las 4 reservas) **no** aparece como inactivo en Y, mientras que el mismo escenario en X **sí** lo marca inactivo. Es una decisión de diseño explícitamente documentada y justificada (no es invención silenciosa), pero se aleja del literal de la pregunta MVP #4 (“sin asistir”).

Defectos confirmados

1. Capacidad de sesión igual a cero se incluye silenciosamente en la media de ocupación sin exclusión ni limitación declarada (contradice AC-03).
2. Un rango de fechas explícito (YYYY-MM-DD a YYYY-MM-DD) en la pregunta se descarta en silencio y se sustituye por “este mes”, con un mensaje de aclaración que es inexacto (“no se especificó periodo” cuando sí se especificó).
3. (Menor, de modelo de dominio) `ClassSession` no tiene ningún campo de cancelación a nivel de sesión — no existe forma de representar “el box canceló esta clase”, a diferencia de `X`. Esto limita qué se puede modelar en el mock y en un futuro adaptador real frente al campo `Calendar.cancelled` que la propia documentación de `Y` identifica como existente (aunque ambiguo) en la API real.

Afirmaciones no verificables o parcialmente sobrecargadas

- `docs/07_Acceptance_Evaluation.md` afirma que la revisión de código fue de “alto esfuerzo, 7 ángulos de búsqueda” y que los 10 hallazgos fueron “verificados CONFIRMADOS por ejecución directa de código”. Esa revisión sí encontró y corrigió defectos genuinos (verificado: los tests de regresión citados existen y pasan), pero el proceso descrito no detectó los dos defectos que sí encontré yo. La afirmación de rigor del proceso de revisión es válida en lo que cubrió, pero no debe leerse como “cobertura exhaustiva de casos límite” — no lo es.
- La declaración de la propia rúbrica de aceptación de que “las 57 pruebas respaldan genuinamente las afirmaciones de aceptación” (AC-09) es cierta para lo que las pruebas cubren, pero no es una prueba de ausencia de defectos fuera de ese conjunto, como demuestra este informe.

Fortalezas

- Cultura de transparencia notable: `docs/07_Acceptance_Evaluation.md` documenta una revisión de código interna que encontró y corrigió 8 defectos reales (por ejemplo, una comparación de periodos que antes mezclaba silenciosamente métricas distintas, o una caída al reloj del sistema en dataset vacío) con tests de regresión dedicados para cada uno — es una práctica de calidad genuinamente buena y poco habitual verla documentada con este nivel de honestidad.
- Documento de descubrimiento de API (`03_API_Discovery.md`) extremadamente preciso: identifica con cita de página el hallazgo más importante de todo el proyecto (no existe ningún endpoint que devuelva reservas acotadas por fecha/franja), y conecta ese hallazgo directamente con la decisión arquitectónica D4 y con el hand-off. También identifica correctamente que `GET /clients/no-booking/:date` da soporte directo a la pregunta de usuarios inactivos.
- Batería de test más amplia y con más pruebas de integración de extremo a extremo contra el dataset real committeado (no solo contra fixtures de mano), incluyendo aserciones de negocio concretas (p. ej., “Movilidad es la clase con menor ocupación este mes”).
- Separación puerto/adaptador (DataSource Protocol) limpia y usada de forma idéntica en producción y en tests.

Riesgos residuales

- Los dos defectos confirmados arriba siguen presentes en el paquete entregado; no están mencionados en `docs/06_Handoff.md` como limitaciones conocidas, así que un desarrollador nuevo no sabría de su existencia sin leer este informe.
- El fixture de datos (2.3 MB, 40 clientes / 955 sesiones / ~8000 reservas, 180 días de historial) es notablemente grande para un MVP cuyo objetivo es demostrar 5 preguntas

de negocio; aporta valor de test de integración a gran escala, pero es desproporcionado si el criterio es “tamaño de fixture proporcional al MVP”.

- El motor de lenguaje natural no reconoce rangos de fecha explícitos en absoluto (a diferencia de X), lo que limita la utilidad práctica de “pregunta con dos fechas concretas” — un caso de uso razonable para un propietario de box.

Veredicto de reproducibilidad

Reproducible con una salvedad menor de entorno. Cero dependencias de terceros en tiempo de ejecución, dataset regenerable byte a byte, semilla y fecha ancla fijas en código (no en el reloj del sistema). La única fricción encontrada fue que el intérprete de Python declarado como mínimo (3.11) no estaba disponible en el entorno de evaluación (solo 3.10); el proyecto funcionó de todas formas sin cambios, así que no bloqueó la reproducción, pero conviene señalar la discrepancia.

3. Veredicto comparativo

Candidato más fuerte: candidate-x, por una diferencia que considero **material** (92 vs. 79, 13 puntos), concentrada casi enteramente en la dimensión con más peso de la rúbrica: corrección funcional (25 puntos posibles).

La razón no es que candidate-y esté mal construido — su documentación de descubrimiento de API, su cultura de revisión de código transparente y su cobertura de tests son, en varios aspectos, tan buenas o mejores que las de X. La diferencia decisiva es que **mi propia batería adversarial, reproducida de forma independiente, encontró dos defectos de producto reales y actualmente presentes** en candidate-y (capacidad cero mezclada sin avisar; rango de fechas explícito ignorado con un mensaje de aclaración falso) que no existen en candidate-x bajo los mismos escenarios. Ambos son exactamente los tipos de caso límite que la propia rúbrica pide comprobar (“capacidad igual a cero”, “redacción de periodo ambigua”), y ambos violan directamente el requisito transversal de explicabilidad (US-06/AC-03: declarar qué datos se excluyeron, no afirmar cosas falsas sobre lo que el usuario pidió).

Además, candidate-x cumple el literal exacto de la pregunta MVP #4 (“usuarios sin *asistir*”) usando asistencia real como criterio, mientras que candidate-y —de forma documentada y razonada, no oculta— responde a una pregunta ligeramente distinta (ausencia de *reserva*). Esa es una decisión defendible dado lo que la API real permite, y no la penalizo como “invención silenciosa”, pero sí como una respuesta menos fiel al enunciado literal del MVP en la dimensión de corrección funcional.

En las dimensiones de determinismo, arquitectura, comprensión de la API y documentación, ambos candidatos están muy cerca (diferencias de 1-2 puntos), y en algún matiz concreto (profundidad del hallazgo central sobre agregación de reservas, transparencia del proceso de revisión) candidate-y iguala o supera a candidate-x. Si la pregunta fuera solo “¿cuál entiende mejor la API y documenta mejor sus supuestos?”, el resultado sería un empate técnico. La diferencia material aparece específicamente al **ejecutar y presionar el código real**, no al leer su documentación.

¿Es la diferencia material?

Sí. Trece puntos sobre 100, con el grueso concentrado en la dimensión de mayor peso (corrección funcional) y respaldado por defectos reproducibles y no por apreciaciones subjetivas, es una diferencia que yo consideraría suficiente para preferir candidate-x como base de producto, aunque recomendaría exigir a candidate-y una ronda de corrección de los dos defectos

encontrados antes de descartarlo — su arquitectura y su cultura de documentación son sólidas y esos dos defectos son, en principio, arreglables sin rediseño.

Dónde ningún candidato es satisfactorio

- **Integración real con AimHarder:** ambos documentan honestamente que la API pública no expone un endpoint que devuelva reservas acotadas por fecha o franja horaria, lo que obliga a un patrón N+1 o a recorrer el histórico completo de cada cliente. Ninguno de los dos resuelve esto (ni podían, dado que está fuera del alcance del MVP); ambos lo señalan correctamente como riesgo de integración futura, sin fingir que está resuelto.
- **Cobertura de lenguaje natural:** ambos son analizadores deterministas basados en patrones, explícitamente acotados a las cinco preguntas del MVP y sus parafraseos documentados. Ninguno es un NLU general, y ambos lo declaran así; cualquier pregunta de negocio nueva requerirá trabajo manual de extensión en los dos casos.
- **Ambigüedad del campo cancelled / waitlist_count de la API real:** ambos documentan que estos campos están mal especificados en la documentación oficial (tipo inconsistente, semántica no aclarada) y ninguno puede resolverlo sin acceso real a la API, correctamente fuera de alcance para este experimento.

Confianza y límites de esta evaluación

- Confianza alta en los hallazgos de hard-failure, integridad de hashes, resultados de test suite y en los dos defectos confirmados de candidate-y: todos están respaldados por ejecución real y reproducible (código disponible en `eval_workspace/adversarial/x_adversarial.mjs` y `y_adversarial.py`, ejecutable de nuevo por cualquiera).
- Confianza media en la puntuación exacta punto a punto de cada dimensión: la rúbrica pide juicio cualitativo (p. ej. “complejidad proporcional al MVP”, “documentación suficiente”) que admite un margen razonable de desacuerdo entre evaluadores; el ranking relativo (X por delante de Y, y por qué) es más robusto que el número exacto de puntos.
- Límite reconocido: no se dispuso de un intérprete Python 3.11 en el entorno de evaluación (solo 3.10), por lo que la conformidad exacta con el mínimo declarado por candidate-y no pudo verificarse al pie de la letra, aunque el comportamiento observado en 3.10 fue correcto en todos los casos probados.
- Límite reconocido: la batería adversarial es amplia pero no exhaustiva; cubre todos los casos que pide explícitamente `EVALUATION_RUBRIC.md` más algunos adicionales motivados por lo que encontré al leer el código (p. ej., la ausencia de campo cancelled a nivel de sesión en Y). Es razonablemente probable que existan otros casos límite no explorados en ninguno de los dos candidatos.
- No se ha intentado ni se ha necesitado inferir la identidad, modelo o herramienta que generó ninguno de los dos candidatos; el análisis se basa exclusivamente en el contenido de los paquetes, en la rúbrica y en evidencia reproducida por el propio evaluador.